

Finding the Successor of a Given node in a BST

To find the in-order successor of a given node in a Binary Search Tree (BST)—which is the node with the next larger value—there are two main cases to consider.

Case 1: The Node Has a Right Subtree

- **What to do:** Go to the right child once, and then keep going **left** as far as possible.
- **Why:** In a BST, values in the right subtree are larger than the current node. The smallest value among all those larger values is the leftmost node in that right subtree.
- **Example:** If a node has a right child 15, and 15 has a left child 12 (with no further left children), 12 is the successor.

Case 2: The Node Has No Right Subtree

- **What to do:** Travel **up** the tree using parent pointers (or from the root if searching without parent pointers) to find an ancestor.
- **The specific target:** Stop at the lowest ancestor for which the current node (or sub-path) lies in its **left** subtree. In other words, look for the first parent where you take a **left turn** when climbing up from the node.
- **Edge Sub-case (No Successor):** If you reach the root without ever taking a left turn (meaning the node is the largest value/rightmost node in the entire tree), the in-order successor does not exist, and the result is null or -1.

To find the **in-order predecessor** of a given node in a Binary Search Tree (BST), there are **two main cases** based on the structure of the tree around that node.

The in-order predecessor is the node that comes immediately before the given node in an in-order traversal (the node with the largest value that is smaller than the target node).

Case 1: The Node Has a Left Subtree

- **Condition:** If the given node has a non-empty left child/subtree.
- **How to find it:** The predecessor is the **maximum value in that left subtree**.
- **Action:** Start at the left child of the given node, then go as far **right** as possible until you reach the rightmost leaf of that left subtree.

Case 2

: The Node Does Not Have a Left Subtree

- **Condition:** If the left child of the given node is NULL.
- **How to find it:** The predecessor is one of the node's **ancestors**.

- **Action:** Traverse up the tree from the root (or track it while searching for the node) to find the **nearest ancestor** for which the given node (or subtree) lies in its **right subtree**. In other words, it is the last ancestor you turned *left* from while moving down, or the last ancestor where you went right to reach the current path. If no such ancestor exists (meaning the node has the absolute smallest value in the BST), the predecessor is NULL.

To find the predecessor and successor of a given node in a Binary Search Tree (BST), use the structural properties of the tree to track values during a downward search or navigate via subtrees and parent pointers. [\[1, 2\]](#)

An **inorder predecessor** is the largest value smaller than the target key. An **inorder successor** is the smallest value greater than the target key. [\[1, 2, 3\]](#)

Finding the Successor

There are two main cases to find the successor of a node with a given key: [\[1\]](#)

1. **If the node has a right subtree:**
 - The successor is the **minimum value** (leftmost node) in that right subtree. [\[1, 2\]](#)
2. **If the node does not have a right subtree:**
 - Traverse down from the root while comparing values.
 - Keep a temporary pointer (successor) that updates whenever you move to the *left* child (because that ancestor is greater than your target). The last recorded value when the target is found or reached is the successor. [\[1\]](#)

Finding the Predecessor

The logic for the predecessor is symmetric: [\[1\]](#)

1. **If the node has a left subtree:**
 - The predecessor is the **maximum value** (rightmost node) in that left subtree. [\[1\]](#)
2. **If the node does not have a left subtree:**
 - Traverse down from the root.
 - Keep a temporary pointer (predecessor) that updates whenever you move to the *right* child (because that ancestor is smaller than your target).

Complexity

- **Time Complexity:** $O(H)$, where (H) is the height of the tree (proportional to the depth of the node).
- **Space Complexity:** $O(1)$ for iterative approaches.

